

d064_clustering_model-selection

DRAFT — auto-generated from Markdown source. Pending a professional slide design; content is accurate but formatting is placeholder.

Clustering Model Selection

Two complementary techniques for the analyst who has already fit a clustering model and now wants to control how it is judged, and how its hyperparameters are chosen — mirroring the model-selection decks already produced for classification (d048) and regression (d055).

2 TECHNIQUES • CUSTOM EVALUATION • CLU_TUNE

Eduardo Ogasawara eduardo.ogasawara@cefet-rj.br <https://eic.cefet-rj.br/~eogasawara>

Two Different Questions

- **Custom evaluation** answers: *which metrics should judge this partition?*
- **clu_tune** answers: *which hyperparameter configuration should I keep?*

Both keep the exact same clustering workflow — configure, fit, cluster, evaluate — and change only what happens around it, not the workflow's shape.

Customizing Evaluation Metrics

Every clustering model in `daltoolbox` carries default internal and external evaluation metrics from its class constructor. Those defaults can be inspected and replaced when the analyst wants a narrower or different evaluation view.

```
model <- cluster_kmeans(k = 3)
model$eval_internal <- list(model$clu_utils$metric_silhouette)
model$eval_external <- list(model$clu_utils$metric_entropy)

model <- fit(model, x)
clu <- cluster(model, x)
evaluate(model, clu, ref)
```

- `eval_internal` : metrics computed from the data and the partition alone (no reference labels needed) — e.g. `silhouette` , which summarizes cohesion and separation in one number.
- `eval_external` : metrics computed against a reference partition, when one exists — e.g. `entropy` , which shows how mixed each cluster is relative to the reference.
- The available metric functions can be listed directly: `grep("^metric_", names(model$clu_utils), value = TRUE)` .
- What changes is only the *content* of `eval_internal` / `eval_external` — the base `clusterer()` class defines sensible common defaults, and concrete classes can override them.

References: MacQueen, J. (1967). *Some Methods for Classification and Analysis of Multivariate Observations*. — Rousseeuw, P. J. (1987). *Silhouettes*. — Shannon, C. E. (1948). *A Mathematical Theory of Communication*.

Hyperparameter Tuning — `clu_tune`

Selects hyperparameters for a clustering method by wrapping a base model and searching a configuration range — the same tuning idea already familiar from `cla_tune` and `reg_tune`, applied to unsupervised learning.

```
base_model <- cluster_kmeans(k = 2)
base_model$metric <- base_model$clu_utils$metric_silhouette
base_model$selector <- base_model$clu_utils$selector_best
base_model$eval_internal <- list(base_model$clu_utils$metric_silhouette)
base_model$eval_external <- list(base_model$clu_utils$metric_entropy)

model <- clu_tune(base_model, ranges = list(k = 2:10))
model <- fit(model, x)
model$k
```

- Wraps any base clustering model the same way `cla_tune` / `reg_tune` wrap a supervised learner.
- The example tunes `k` against a compact internal criterion (`silhouette`) while still reporting an external interpretation score (`entropy`) — a didactic simplification, not a requirement.
- `model$selector` picks the winning configuration from the range once each candidate has been scored.

References: Satopaa, V., Albrecht, J., Irwin, D., and Raghavan, B. (2011). *Finding a "Kneedle" in a Haystack*. — Rousseeuw, P. J. (1987). *Silhouettes*.

What to Observe

- Evaluation customization and hyperparameter tuning are two separate, composable concerns — you can change one without the other.
- Both examples keep configure → fit → cluster → evaluate intact; tuning changes *how the configuration is chosen*, not how the experiment is read.